

Nine systems, and the decisions behind them

Elena Revicheva · AI Automation Engineer · AI Solutions Architect

[aideazz.xyz/portfolio](#) · [aideazz.xyz/api](#) · [aideazz.xyz/ai-ops-wiki.html](#) · [atuona.xyz](#)

I design, build and operate production AI automation alone: a fleet of LLM agents on one Oracle Cloud server, controlled from Telegram, with HubSpot as the system of record. Each system replaced work I was doing by hand. This document explains why each one exists, the design decisions that mattered, and what is running today. It is written for an engineering team deciding whether that judgement belongs on their roadmap.

1,640	534	500	8	22
COMMITTS · OPS AGENT	EVAL TESTS · JOB ENGINE	API AUDITS SERVED	AI FILMS, AUTO-EDITED	PUBLIC INCIDENT WRITE-UPS

How I build: AI-native, and that is the point

I frame the problem, design how the pieces fit, and delegate the implementation to coding agents (Claude Code and Cursor). Then I prove that what came back is true. On several repos most commits carry the agent's authorship, and I say so because anyone can see it in a `git log`. The scarce skill has moved from typing the code to **knowing which output to distrust and proving the difference**. Every discipline in Part 3 exists because generated code is confidently wrong in specific, repeatable ways: it reports success it did not achieve, it writes a second implementation instead of finding the first, and it produces plausible numbers.

Before AI I spent seven years as Deputy CEO and Chief Legal Officer of a national e-government operator. So I start from the business process and the compliance risk, not from the tool.

PART 1 · THE SHARED SPINE

LAYER	CHOICE	WHY THIS, AND NOT THE OBVIOUS ALTERNATIVE
Compute	One Oracle Cloud server, PM2	Failures are observable in one place. GitHub Actions deploys named files to 12 services and verifies the fleet afterwards.
Control	Telegram, not a web dashboard	Approvals happen with one tap, wherever I am. 110+ commands, including a coding assistant that opens real pull requests.
Ledger	HubSpot CRM	Every agent writes with a prefix that names its author, so provenance is a filter rather than an investigation.
Models	Five providers, three ranked chains	One API key is a single point of failure for the whole operation. The chains are ordered by use case: quality, classification, bulk.
Language	TypeScript and Python	TypeScript wherever money or CRM state is touched, so the type checker catches a class of mistake before deploy. Python where the ML ecosystem lives.

The decision I would defend hardest: the five-provider chain. On 19 Aug 2026 the primary provider's balance hit zero, and the chain fell through in 2.4 seconds with no lost request. The lesson that took longer: the order must differ by use case. Reasoning models return empty strings under low token budgets, so a classification chain led by one *fails silently*, which is worse than an error.

PART 2 · THE NINE SYSTEMS

1 · CTO AIPA: operations brain and inbound lead concierge

TypeScript · HubSpot API · Telegram · Resend · GitHub API · 1,640 commits

Problem. Inbound needed a fast, correct reply with a human approving it, and overnight failures across the fleet were invisible until morning.

Design. A form submission becomes a CRM record and a drafted reply, which arrives as a one-tap approval card. Delivery and open events are written back to the deal. Two systems can write the reply, and redundancy alone means the faster one always wins, so a **cached health verdict decides precedence** (5-minute grace period, 20-minute safety net). The same bot is a coding co-pilot: `/code` and `/fix` open real PRs, `/approve` keeps a human on the last click, and `/decision` and `/debt` record architecture choices.

Running. An 08:00 briefing every day. A synthetic lead goes through the real pipeline every morning, silent on pass and alerting on failure.

2 · VibeJobHunter: job discovery with a human on the last click

Python · LangGraph · SQLite checkpointing · pytest · Perplexity API · 578 commits

Design. A LangGraph state machine: gate → score → route → notify. It is checkpointed, so a crash resumes instead of restarting, and there is a deliberate human-approval interrupt. An LLM judge scores fit, and a **trust clamp** limits how far keyword rules can overrule it (this fixed an incident where keyword bonuses silently outvoted a correct "no"). A feedback loop writes my real CRM decisions, and my typed reasons, back into the judge. It **fails closed** on an LLM outage. I removed auto-apply after measuring **volume and zero outcomes**.

New in September. The Perplexity API researches every queued job, with sources cited and results cached. A tailored cover letter and a lane-matched CV go onto every deal. This week a **read-back audit** showed only 11 of 23 deals really complete: a refused API call was being logged as "no data". I fixed it with retry-on-429 and failed lookups that raise, then added a self-healing job every 2 hours and a daily audit with alerts. Result: 19 of 23, and the other 4 proven to be closed postings.

Harness. 534 tests. The offline layers run with the AI disabled and no network, so they are cheap enough to run before every scoring-rule change. One test class covers bias compensation.

3 · AI Visibility Audit API: a public product

TypeScript · REST · Oracle Cloud · aideazz.xyz/api

Problem. Buyers and recruiters now ask an AI assistant before a search engine, and nobody could tell a business whether ChatGPT, Perplexity or Claude can read its site.

Design. 34 checks per URL (answer-engine readiness, structured data, crawlability, technical SEO). Each one returns the fix and why it matters, not only a score. The public page states production figures as **floors that only go stale in the safe direction**: a running total printed exactly is wrong by tomorrow.

Running. 500 audits served across 248 URLs, counted today from the log line the API writes for each audit. My own hub scores 100/100. The API also has a promo film cut by the same automated edit pipeline as system 6.

4 · Publishing engine: blog, podcast, social

TypeScript · Dev.to canonical · Buffer · daily at 14:30 Panama

Design. Grounded generation. Before writing, the generator collects production evidence: uptimes, real commit messages, CRM counts. A gate rejects any figure it cannot trace, then retries up to 4 times with a different angle each time and salvages the sourced paragraphs. If the model still decorates the facts, a measured-notes article is assembled deterministically. Detection is token-overlap against everything already published, which caught near-duplicates that exact-slug matching missed.

5 · AI Ops Wiki: the public incident record

generated from structured entries, never hand-edited · ai-ops-wiki

22 incidents and 19 named failure modes, each with the verified number that proves it. Examples: *redundancy is not precedence*, *acknowledgement is not completion*, *the prompt is a source*, *vacuous guard*. This is the fastest way for a stranger to judge how I think, including the times a system fooled me.

6 · Atuona: an automated AI film studio

TypeScript · ffmpeg · multi-provider video and image APIs · atuona.xyz

Problem. Most AI video tools stop at generating a clip. Pacing, voice timing, transitions, text and the mix are still done by hand. I automated that second half.

This week. Films #7 and #8 were published on 22 Sep, so the gallery now holds **8**. Film #8 is 3:36 long and cut from 16 shots, all newly generated. The engines were chosen per shot type through a **bake-off** (one for close-ups, another for wides) behind a generation tool with a **hard budget guard**. Film #7 turned still images into moving shots with a **depth-based 2.5D camera**. The bot now drives a dozen video and image engines from one command, with an LLM acting as director.

Design that transfers. Voiceover is locked to its clip by a duration formula. Short clips get slow motion, never a held frame. Crossfades run on a running-offset accumulator. Music ducks under the voice with sidechain compression, and the mix is normalised to broadcast loudness (−16 LUFS; film #8 measured −15.7). Every render is verified frame by frame before publishing.

Two incidents from the same day. (1) New engine lines pushed a menu past the chat platform's 4,096-character limit, so the bot silently answered nothing. The fix was chunked replies and a rule to re-measure on every new menu line. (2) A pre-render **spending cap could never fire**: it read price fields the vendor never sends, so every price passed as `NaN`. I fixed it and published it as the *vacuous guard* concept. The rule: print the vendor's raw response once before trusting any guard built on its shape.

7 · Espaluz: bilingual expat advisor on WhatsApp and Telegram

Python · LangChain · pgvector RAG on Oracle Autonomous DB · voice and image in · 613 commits

Born from my own family's relocation to Panama. Two layers of memory: short-term conversation context, plus a **per-family vector store retrieved by relevance rather than recency**. The answer in November depends on the form a family filled in in June. Voice in and out, image understanding for forms, and payments wired for trials. Design constraint: emotional intelligence before grammar correction.

8 · Atlas: radar for advertising angles

inside the ops agent · 7 verticals · weekly

Reads public ad libraries and reports which creative angle is running in a market, and for how long. The rule is **detect, not predict**: "this is live and three weeks old" can be checked, while "this will work" cannot. Outcomes are written back to the CRM and scored.

9 · DragonTrade and Algom

JavaScript · 176 commits · paper trading only

DragonTrade is a trading-education agent that runs on paper trading only. I built it to learn how to operate an agent that must **never take an irreversible financial action without a human**. **Algom** listens to a public post stream for buying intent and files leads under its own CRM prefix, so its yield can be measured against every other source.

PART 3 · THE DECISIONS THAT REPEAT

Measure at the edge, not the source	A commit is not a deploy, and an accepted email is not a delivered one. Verify where the request actually lands.
Reconcile: read the end state back	A job's own log is not proof. Audits read the CRM back and alert on any gap.
Unmeasured is not zero	A tracker exits non-zero when it cannot measure. A refused call is a failure, never "no data".
A guard must be able to fire	Test every cap and filter against the vendor's real response shape (the vacuous guard).
Fail open or closed, per system	Alerting fails open: a duplicate beats silence. Publishing and money fail closed.
A human on the last click	Anything that reaches another person, or moves money, waits for one tap.
Every record names its author	Many agents write to one CRM, and a source prefix makes provenance a filter.
Context is an artefact	Two coding agents share one repo through a claim board and handoff blocks, not through memory.

PART 4 · WHAT I WOULD BRING

An engineer who turns a staff team's manual work into agents and automations, takes them to production with CI/CD and monitoring, and treats the correctness of AI output as the main engineering problem rather than an afterthought. I have done all of it alone, without an on-call rota, which forces different decisions from the ones a team makes. I would like to make them inside a team that already has users.

Figures re-read from the running systems and repositories on 23 September 2026; audit counts come from the production log. Code on GitHub. I am happy to walk through any of it live, including the parts that broke. · Elena Revicheva · aideazz.xyz/portfolio